BITÁCORA - PRÁCTICA 9: LA MAZMORRA (Encapsulamiento, primera parte

Integrantes:

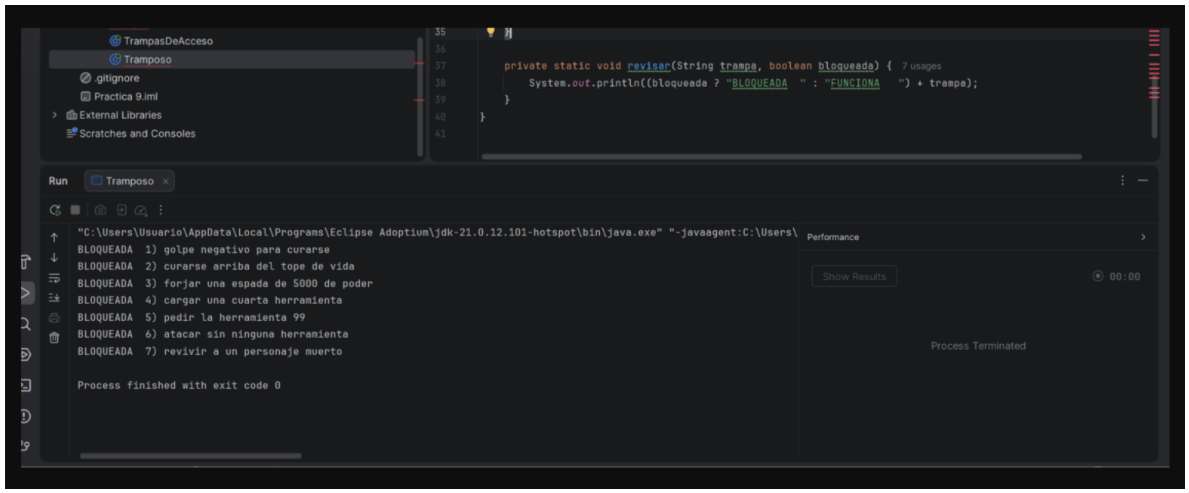
Isaac Cruz Chávez (Número de cuenta:424020705)

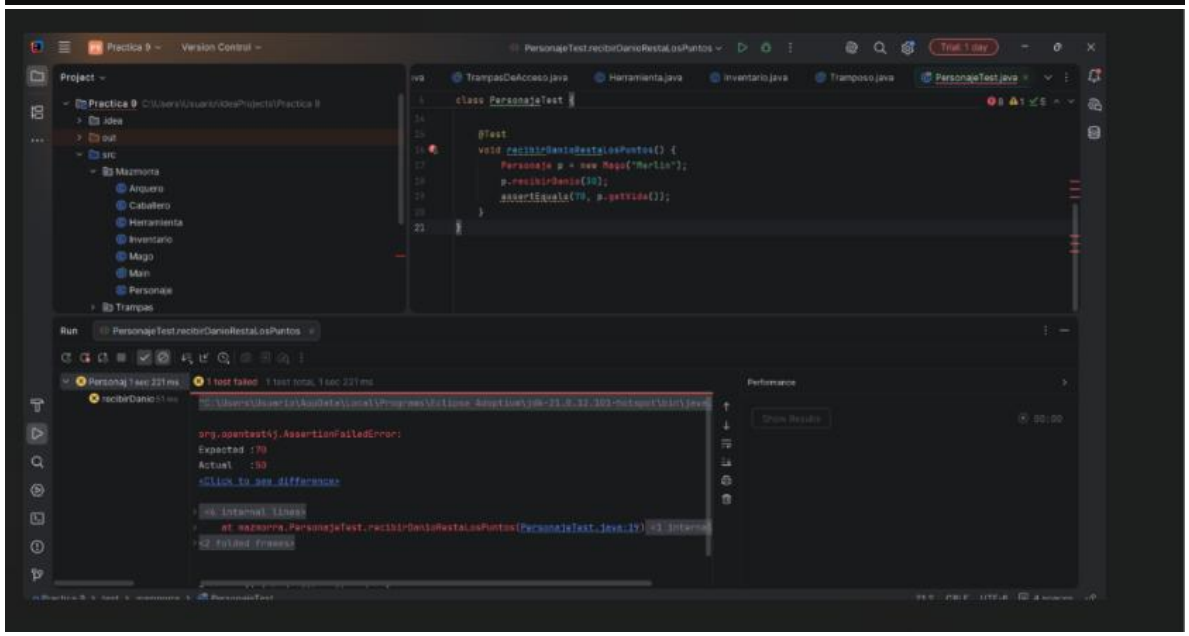
Grupo: 6

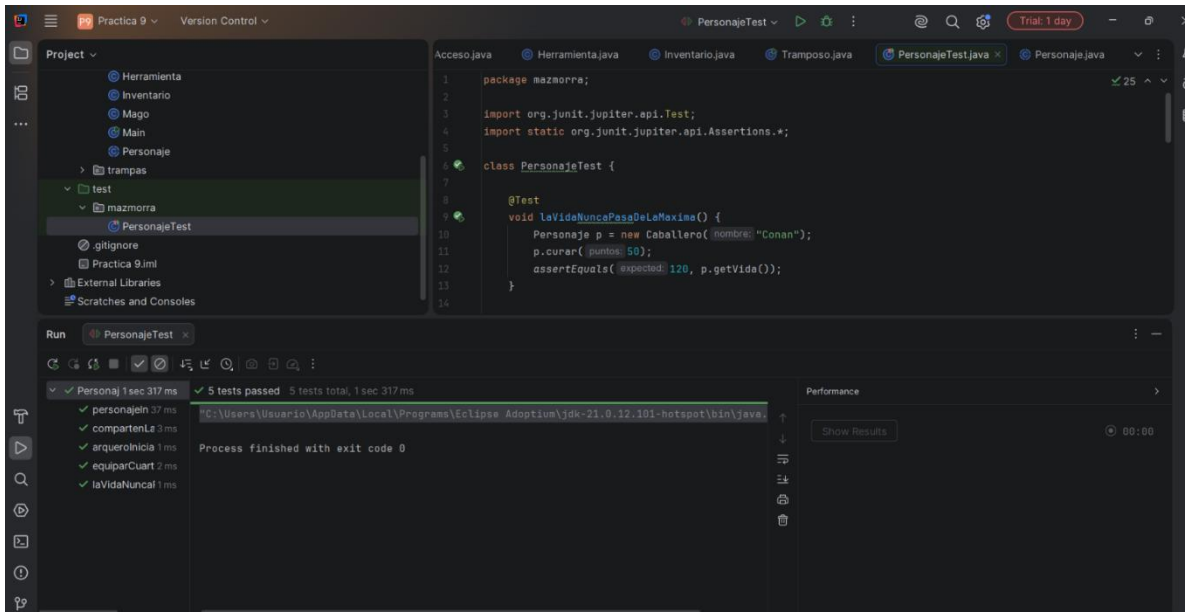
Materia: Modelos de Programación Orientada a Objetos (2027-1)

- Capturas de Evidencia




TrampasDeAcceso
Tramposo
.gitignore
Practica 9.iml
External Libraries
Scratches and Consoles
private static void revisar(String trampa, boolean bloqueada) { 7 usages
System.out.println((bloqueada ? "BLOQUEADA " : "FUNCIONA ") + trampa);
Run
Tramposo
"C:\Users\Usuario\AppData\Local\Programs\Eclipse Adoptium\jdk-21.0.12.101-hotspot\bin\java.exe" "-javaagent:C:\Users\
BLOQUEADA 1) golpe negativo para curarse
BLOQUEADA 2) curarse arriba del tope de vida
BLOQUEADA 3) forjar una espada de 5000 de poder
BLOQUEADA 4) cargar una cuarta herramienta
BLOQUEADA 5) pedir la herramienta 99
BLOQUEADA 6) atacar sin ninguna herramienta
BLOQUEADA 7) revivir a un personaje muerto
Process finished with exit code 0
Performance
Show Results
00:00
Process Terminated


Practica 9
Version Control
Project
Practica 9
.idea
out
src
Mazmorra
Arquero
Caballero
Herramienta
Inventario
Mago
Main
Personaje
Trampas
TrampasDeAcceso.java
Herramienta.java
Inventario.java
Tramposo.java
PersonajeTest.java
class PersonajeTest {
@Test
void recibirDanioRestaLosPuntos() {
Personaje p = new Mago("Merlin");
p.recibirDanio(30);
assertEquals(70, p.getVida());
}
}
Run
PersonajeTest.recibirDanioRestaLosPuntos
1 test failed
org.opentest4j.AssertionFailedError:
Expected :70
Actual :99
Click to see difference
Performance



1. Las cuatro relaciones

- **Herencia (extends y super):**
    - *Línea de código:* public class Caballero extends Personaje y super(nombre, 120); dentro del constructor de la clase hija.
    - *Qué le pasa a la parte:* La clase hija hereda todos los atributos y métodos públicos/protegidos del padre. Al usar super(...), se inicializan los atributos de la clase abstracta padre (Personaje); si el objeto principal desaparece, la estructura de la jerarquía conceptual se extingue junto con él.
- **Composición (new interno):**
    - *Línea de código:* this.inventario = new Inventario(); dentro del constructor de Personaje.
    - *Qué le pasa a la parte:* El objeto dueño (Personaje) fabrica y es dueño absoluto de la parte (Inventario). Si el personaje desaparece, el inventario muere con él porque se construyó internamente y no existe por separado.
- **Agregación (Relación 0..3):**

    - *Línea de código:* public boolean agregar(Herramienta h) en Inventario.
    - *Qué le pasa a la parte:* La parte (Herramienta) se construye por fuera de la clase y se pasa como parámetro. Por lo tanto, si el inventario o el personaje desaparecen, la herramienta sobrevive ya que pertenece a un ámbito externo.
- **Asociación (Relación 0..1 de compañero):**
    - *Línea de código:* private Personaje companero; conectado mediante setCompanero(Personaje companero).
    - *Qué le pasa a la parte:* Se conocen y colaboran (por ejemplo, para curarse con ayudarCompanero()), pero ninguno es parte del otro; ambos objetos sobreviven de manera independiente en la memoria.

2. Los huecos y el encapsulamiento

- **¿Funcionó alguna trampa la primera vez?**

Ninguna trampa funcionó gracias al encapsulamiento estricto. Por ejemplo, la trampa 1 (*golpe negativo para curarse*) fue bloqueada directamente por la validación if (puntos > 0) implementada en los métodos recibirDanio y curar de la clase Personaje.

- **¿Por qué el diagrama no tiene setVida ni getHerramientas?**

No se incluyen para proteger la integridad de los datos mediante el encapsulamiento. Si existiera un setVida, cualquier clase o jugador externo podría modificar la vida de forma descontrolada (por ejemplo, asignándose 9999 puntos). Del mismo modo, si se expusiera el arreglo completo con un getHerramientas, se podría insertar una cuarta herramienta o alterar el inventario sin pasar por el método regulado agregar.

3. Una prueba roja

- **Mensaje de error copiado:**

org.opentest4j.AssertionFailedError: expected: <70> but was: <50> *(en la línea de la prueba recibirDanioRestaLosPuntos)*.

- **Explicación:**
    - **expected: <70>:** Representa el valor numérico que la prueba *esperaba* recibir de forma anticipada.
    - **actual: <50>:** Representa el resultado que el código del programa calculó y devolvió *realmente* de manera efectiva.
    - **¿Quién estaba mal?** Estaba mal la prueba unitaria. Un Mago se inicializa con una vida máxima de 80 puntos; al aplicarle un daño de 30, la lógica matemática correcta arroja un resultado de 50 de vida restante ($80 - 30 = 50$). El código principal calculó el 50 de forma correcta, pero la aserción de la prueba exigía erróneamente el valor de 70.